Exercise 2

Alex WIlson
Introduction to Statistical Methods

January 24, 2021

Abstract

Galileo performed many experiments using various kinematic experiments using balls and ramps, recording the heights of the starting points of the balls, and the final distance reached by them. In this exercise Galileo's original results from the 'ski jump' ramp will be used to demonstrate statistical and data fitting methods. Using curve_fit from scipy.optimize the data will be fitted to three different hypotheses, from which parameters will be determined and the quality of each hypothesis will be tested.

1 Data

h (punti)	d (punti)	σ (punti)
1000.0	1500.0	15.0
828.0	1340.0	15.0
800.0	1328.0	15.0
600.0	1172.0	15.0
300.0	800.0	15.0

Table 1: Galileo's data set from 1608[1]. Recorded in units of punti which is slightly less than 1mm.

2 Hypotheses

The data, from 1, was fitted to three hypotheses, equations 1, 2, 3, using the curve_fit function.

$$d = \alpha h \tag{1}$$

$$d = \alpha h + \beta h^2 \tag{2}$$

$$d = \alpha h^\beta \tag{3}$$

The linear fit with a single parameter, equation 1, shown in Figure 1, gave the parameter, α as 1.66 $\pm$ 9 $\cdot 10^{-3}$. The quadratic fit with parameters α and β, equation 2, shown in Figure 2, gave parameters: α as 2.79 $\pm$ 5 $\cdot 10^{-2}$, and β as -1.35 $\cdot 10^{-3}$ $\pm$ 6 $\cdot 10^{-5}$. The final fit, again with two parameters α and β, equation 3, shown in Figure 3, gave parameters: α as 4.38 $\cdot 10^{1}$ $\pm$ 5, and β as 5.11 $\cdot 10^{-1}$ $\pm$ 2 $\cdot 10^{-2}$.

3 Plots

The following are figures were plotted using curve_fit and matplotlib, using the functions shown below

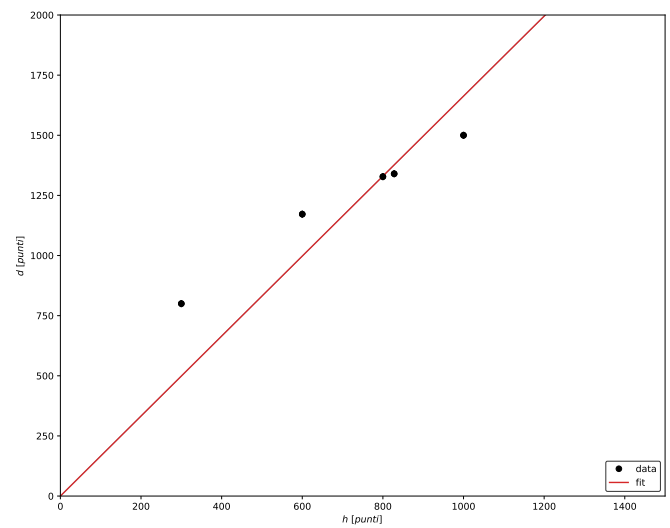

Figure 1: Linear fit for data, of the form of equation 1. As can be seen visually this hypothesis was not a good fit for this data.

```
def func1(h,*theta):
    alpha, = theta
    return alpha*h
p01 = np.ones(1)#initial guess for paramaters theta0
     and theta1
thetahat1,cov1 = curve_fit(func1, h, d,p01 , sig,
    absolute_sigma=True)

def func2(h,*theta):
    alpha, beta = theta
    return alpha*h + beta*h**2
p02 = np.ones(2)#initial guess for paramaters theta0
     and theta1
thetahat2,cov2 = curve_fit(func2, h, d,p02 , sig,
    absolute_sigma=True)

def func3(h,*theta):
    alpha, beta = theta
    return alpha*h**beta
p02 = np.ones(2)#initial guess for paramaters theta0
     and theta1
thetahat3,cov3 = curve_fit(func3, h, d,p02 , sig,
    absolute_sigma=True)
```

Where each function is made of placeholder parameters,*theta, which are then filled with the initial guess for a value that will minimise the χ^2, p0, the curve_fit function then takes these initial guesses and uses them as a starting point to find the actual minimised value of χ^2.

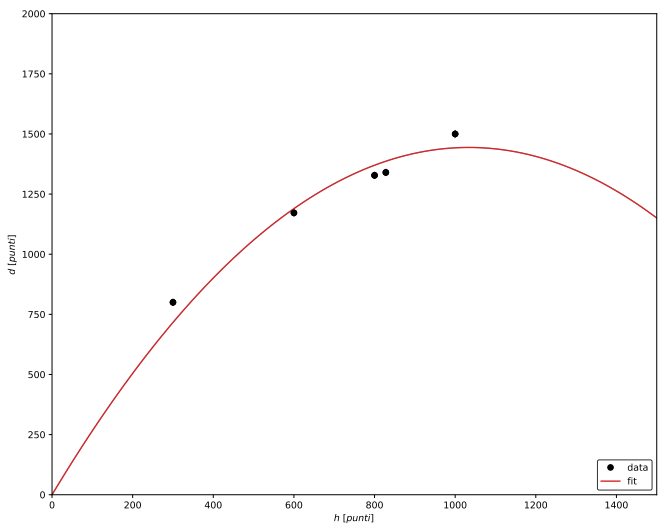

Figure 2: Quadratic fit of form of equation 2. The quadratic fit is a better fit than the linear fit from Figure 1, however it is still not a good fit for the data.

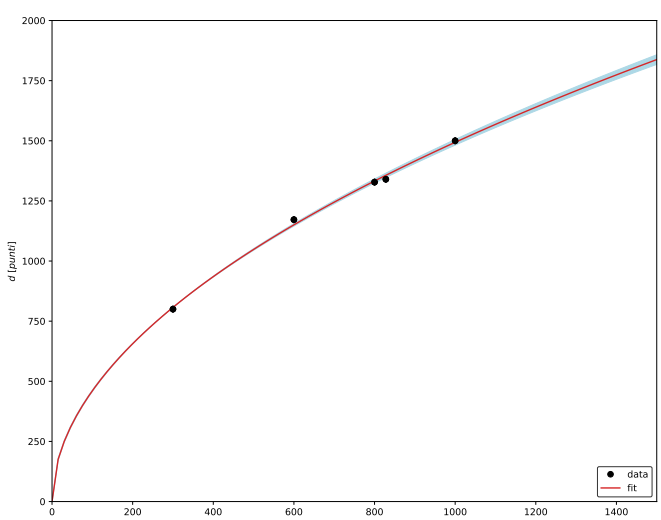

Figure 3: Nonlinear fit of the form of equation 3, as you can see in the plot this is a good fit for the data. The error in the fit is shown by blue shaded region.

4 Goodness of fit

The p-values for each fit were determined using the following code

```
chisq_list1=[]
chisq_list2=[]
chisq_list3=[]
for a in range(len(d)):
    chisq_list1.append(((d[a]-func1(h[a],*thetahat1)
        )/(sig[a]))**2)
    chisq_list2.append(((d[a]-func2(h[a],*thetahat2)
        )/(sig[a]))**2)
    chisq_list3.append(((d[a]-func3(h[a],*thetahat3)
        )/(sig[a]))**2)
chisq1=sum(chisq_list1)
chisq2=sum(chisq_list2)
chisq3=sum(chisq_list3)
print ("p-value for equation 1 = {0}\n".format(chi2.
    sf(chisq1, df=6)))
print ("p-value for equation 2 = {0}\n".format(chi2.
    sf(chisq2, df=5)))
print ("p-value for equation 3 = {0}\n".format(chi2.
    sf(chisq3, df=5)))
```

where the χ^2_{min} values for each fit were calculated gain using the code in lines four to ten. Those values were then used to generate p-values using the chi2 function from scipy.stats.

For equation 1 the χ^2_{min}/n_{dof} was $\frac{661}{6}$, which in turn gave a p-value of $9.81{\cdot}10^{-140}$,which is so small that it means that it is more or less impossible that this fit can accurately describe the data, so the hypothesis can be rejected. This makes sense as the plot for this fit, Figure 1, shows that this fit does not pass near to most of the points.

Equation 2, had a χ^2_{min}/n_{dof} of $\frac{64.7}{5}$, giving a p-value of $1.26{\cdot}10^{-12}$, which again is so small as to make it improbable that this fit can accurately represent the data, however it is significantly more probable than the fit for equation 1, but the hypothesis can be rejected. The inaccuracy of this fit can again be seen in the plot Figure 2.

The final equation, equation 3, had a χ^2_{min}/n_{dof} of $\frac{3.75}{5}$, giving a p-value of 0.59, which makes it more than likely that this fit accurately represents the data, and hence the hypothesis is not rejected.

5 Newtonian Mechanics

Galileo performed this experiment in 1608 [1] before the time of Newtonian mechanics, however now with the aid of such methods, we can derive a model for this experiment. Assuming a solid sphere with moment of inertia equal to $\frac{2}{5}mr^2$, was released from the ramp at height h,on a table of height H, its displacement from the end of the ramp can be calculated, by equating the gravitational potential energy and the kinetic energy, equation 4, and the equation of motion shown in equation 5, to give final relation between d and h shown in equation 6.

$$mgh = \frac{1}{2}mv^2 + \frac{1}{2}I\omega^2 \tag{4}$$

$$d = vt + \frac{1}{2}at^2 \tag{5}$$

$$d = \sqrt{\frac{20}{7}Hh} \tag{6}$$

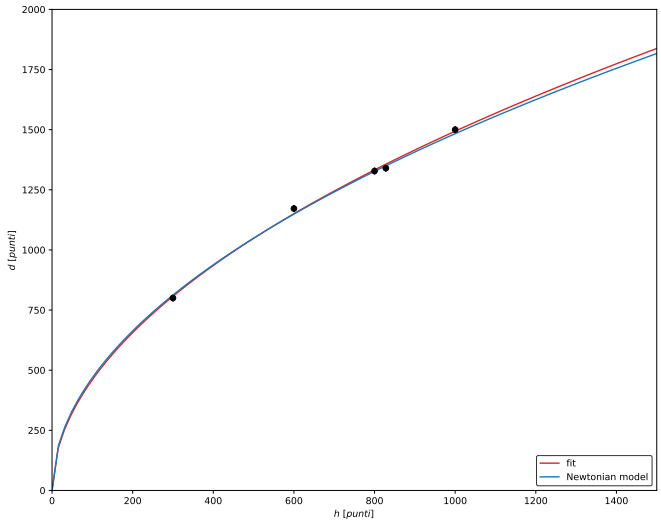

Figure 4: Plot of model derived using Newtonian mechanics (blue), and fit found using least squares method and curve_fit (red). This plot shows that the fit generated using Galileo's data and curve_fit is very close to the model produced using the kinematic equations, provided that the height H of the surface is around 770 punti.

A Python Code

Code used to create plots shown below:

```
'''
==========
  Exercise 2
==========
'''

import matplotlib
import matplotlib.pyplot as plt
import numpy as np
from scipy.optimize import curve_fit
from scipy.stats import chi2
xmin=0
xmax=1500
ymin=0
ymax=2000
h   = np.array([1000, 828,  800,  600,  300])
d   = np.array([1500, 1340, 1328, 1172, 800])
sig = np.array([15,   15,   15,   15,   15])
x_= np.linspace(xmin,xmax,num=100)
'''
=======
Equation 1
=======
'''
def func1(h,*theta):
    '''
    Inputs: x, *theta
    * denotes vector
    Output: linear function with only one parameter,
        alpha
    '''
    alpha, = theta
    return alpha*h
p01 = np.ones(1)#initial guess for paramaters theta0
    and theta1
thetahat1,cov1 = curve_fit(func1, h, d,p01 , sig,
    absolute_sigma=True)
'''
=======
Equation 2
=======
'''
def func2(h,*theta):
    '''
    Inputs: x, *theta
    * denotes vector
    Output: Quadratic function with two parameters,
        alpha and beta
    '''
    alpha, beta = theta
    return alpha*h + beta*h**2
p02 = np.ones(2)#initial guess for paramaters theta0
    and theta1
thetahat2,cov2 = curve_fit(func2, h, d,p02 , sig,
    absolute_sigma=True)
'''
=======
Equation 3
=======
'''
def func3(h,*theta):
    '''
    Inputs: x, *theta
    * denotes vector
    Output: nonlinear function with two parameters,
        alpha and beta
    '''
    alpha, beta = theta
    return alpha*h**beta
p02 = np.ones(2)#initial guess for paramaters theta0
    and theta1
thetahat3,cov3 = curve_fit(func3, h, d,p02 , sig,
    absolute_sigma=True)
def StDev(x,cov,M):
    k=[]
    for i in range(M):
        for j in range(M):
            k.append(x**(i+j)*cov[i,j])
    return (np.sqrt(sum(k)))
'''
=====
Plot 1
=====
'''
fig1 = plt.figure(figsize=(10, 8))
ax1 = fig1.add_subplot(1,1,1)

ax1.plot(h,d,'o',color='black',label='data')
ax1.errorbar(h,d,yerr=sig,fmt='o',color='black')
ax1.plot(x_,func1(x_,*thetahat1),color='tab:red',
    label='fit')

ax1.set_xlabel(r'$h$ $[punti]$')
ax1.set_ylabel(r'$d$ $[punti]$')

ax1.fill_between(x_,func1(x_,*thetahat1)+StDev(x_,
    cov1,1), func1(x_,*thetahat1)-StDev(x_,cov1,1),
    color='lightblue', zorder=-1000)

ax1.legend(frameon='true',edgecolor='black',loc='
    lower right')
ax1.set_xlim(xmin,xmax)
ax1.set_ylim(ymin,ymax)
#3ax1.set_title(r'$Fitted$ $Plot$')
plt.tight_layout()

fig1.savefig("Equation 1 Plot.pdf", format='pdf')



'''
=====
Plot 2
=====
'''
fig2 = plt.figure(figsize=(10, 8))
ax2 = fig2.add_subplot(1,1,1)
```

```
ax2.plot(h,d,'o',color='black',label='data')
ax2.errorbar(h,d,yerr=sig,fmt='o',color='black')
ax2.plot(x_,func2(x_,*thetahat2),color='tab:red',
    label='fit')

ax2.set_xlabel(r'$h$ $[punti]$')
ax2.set_ylabel(r'$d$ $[punti]$')



ax2.fill_between(x_,func2(x_,*thetahat2)+StDev(x_,
    cov2,2), func2(x_,*thetahat2)-StDev(x_,cov2,2),
    color='lightblue', zorder=-1000)

ax2.legend(frameon='true',edgecolor='black',loc='
    lower right')
ax2.set_xlim(xmin,xmax)
ax2.set_ylim(ymin,ymax)
#ax2.set_title(r'$Fitted$ $Plot$')
plt.tight_layout()

fig2.savefig("Equation 2 Plot.pdf", format='pdf')


'''
=====
Plot 3
=====
'''
fig3 = plt.figure(figsize=(10, 8))
ax3 = fig3.add_subplot(1,1,1)

ax3.plot(h,d,'o',color='black',label='data')
ax3.errorbar(h,d,yerr=sig,fmt='o',color='black')
ax3.plot(x_,func3(x_,*thetahat3),color='tab:red',
    label='fit')

ax3.set_xlabel(r'$h$ $[punti]$')
ax3.set_ylabel(r'$d$ $[punti]$')



ax3.fill_between(x_,func3(x_,*thetahat3)+StDev(x_,
    cov3,2), func3(x_,*thetahat3)-StDev(x_,cov3,2),
    color='lightblue', zorder=-1000)

ax3.legend(frameon='true',edgecolor='black',loc='
    lower right')
ax3.set_xlim(xmin,xmax)
ax3.set_ylim(ymin,ymax)
#ax3.set_title(r'$Fitted$ $Plot$')
plt.tight_layout()

fig3.savefig("Equation 3 Plot.pdf", format='pdf')

'''
======
Values
======
'''
t_hats=[thetahat1,thetahat2,thetahat3]
covs=[cov1,cov2,cov3]
stdevs=[]
stdevs2=[]
print(covs)
for i in range(len(t_hats)):
    if i==0:
        stdevs.append(np.sqrt(covs[i][0][0]))
        print('Parameter for equation {0}:\n  alpha
            = {1}+-{2}\n'.format(i+1,t_hats[i][0],
            stdevs[i]))
    elif i>=1:
        stdevs.append(np.sqrt(covs[i][0][0]))
        stdevs2.append(np.sqrt(covs[i][1][1]))
        print('Parameters for equation {0}:\n  alpha
            = {1}+-{2}\n  beta = {3}+-{4}\n
            Covariance = {5}\n'.format(i+1,t_hats[i
            ][0],stdevs[i],t_hats[i][1],stdevs2[i
            -1],covs[i]))

'''
---
p-values
---
'''
chisq_list1=[]
chisq_list2=[]
chisq_list3=[]
for a in range(len(d)):
    chisq_list1.append(((d[a]-func1(h[a],*thetahat1)
        )/(sig[a]))**2)
    chisq_list2.append(((d[a]-func2(h[a],*thetahat2)
        )/(sig[a]))**2)
    chisq_list3.append(((d[a]-func3(h[a],*thetahat3)
        )/(sig[a]))**2)
chisq1=sum(chisq_list1)
chisq2=sum(chisq_list2)
chisq3=sum(chisq_list3)
print('Chisq1',chisq1,'\nChisq2',chisq2,'\nChisq3',
    chisq3)
print('\nChisq1/ndof',chisq1/6,'\nChisq2/ndof',
    chisq2/5,'\nChisq3/nof',chisq3/5)
print ("\np-value for equation 1 = {0}\n".format(
    chi2.sf(chisq1, df=6)))
print ("p-value for equation 2 = {0}\n".format(chi2.
    sf(chisq2, df=5)))
print ("p-value for equation 3 = {0}\n".format(chi2.
    sf(chisq3, df=5)))
'''
=====
Tabel
=====
'''
for o in range(len(h)):
    print('%.1f & %.1f & %.1f \\\\' % (h[o], d[o],
        sig[o]))

'''
=========
Newtonian model
=========
'''
fig4 = plt.figure(figsize=(10, 8))
ax4 = fig4.add_subplot(1,1,1)
H=770
g=9.81
v=np.linspace(0,1600,num=100)
y=np.sqrt(20/7*H*v)
ax4.set_xlim(xmin,xmax)
ax4.set_ylim(ymin,ymax)
ax4.plot(x_,func3(x_,*thetahat3),color='tab:red',
    label='fit')
ax4.errorbar(h,d,yerr=sig,fmt='o',color='black')
ax4.plot(v,y,color='tab:blue',label='data')
```

2a.py

References

[1] G. Cowan. *Introduction to Statistic Methods*. Royal Holloway, University of London. 2021.